

AI in images and video processing

Thomas de Jaeger
thomas.de-jaeger@epita.fr

Object Detection & Optical Character Recognition

Session 5

Object Detection & Optical Character Recognition

Session 5

Objectives:

- Understand how to capture and process video with OpenCV
- Learn basic object detection techniques (Haar, DNN)
- Understand real-time detection with YOLO
- Perform OCR using Tesseract

Introduction to Video Processing

What is a video?

- A video is a sequence of frames (images) shown rapidly, typically 24–60 fps
- Each frame can be processed individually using image processing or ML
- Applications include surveillance, smart cities, autonomous vehicles, live face or object detection



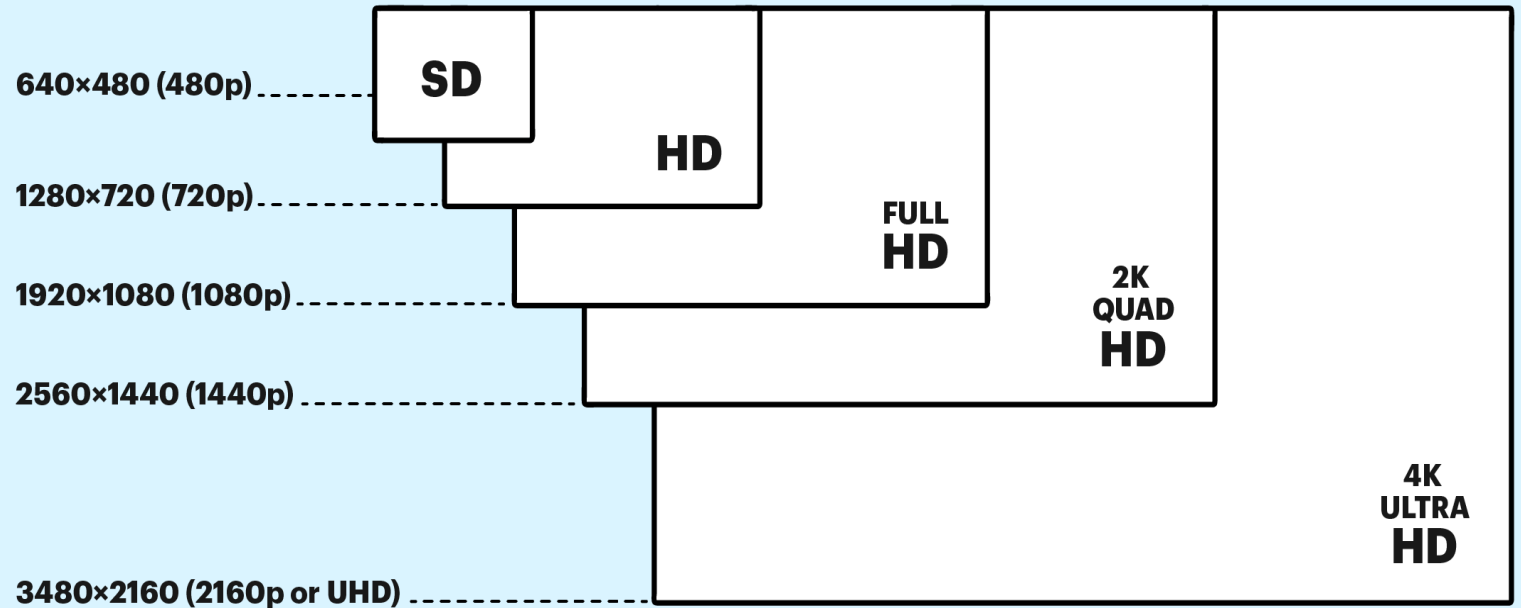
Introduction to Video Processing

Video resolution?

Video resolution is the size of each image in the video.

It doesn't need to be a standard size, but there are common sizes for video.

Resolutions

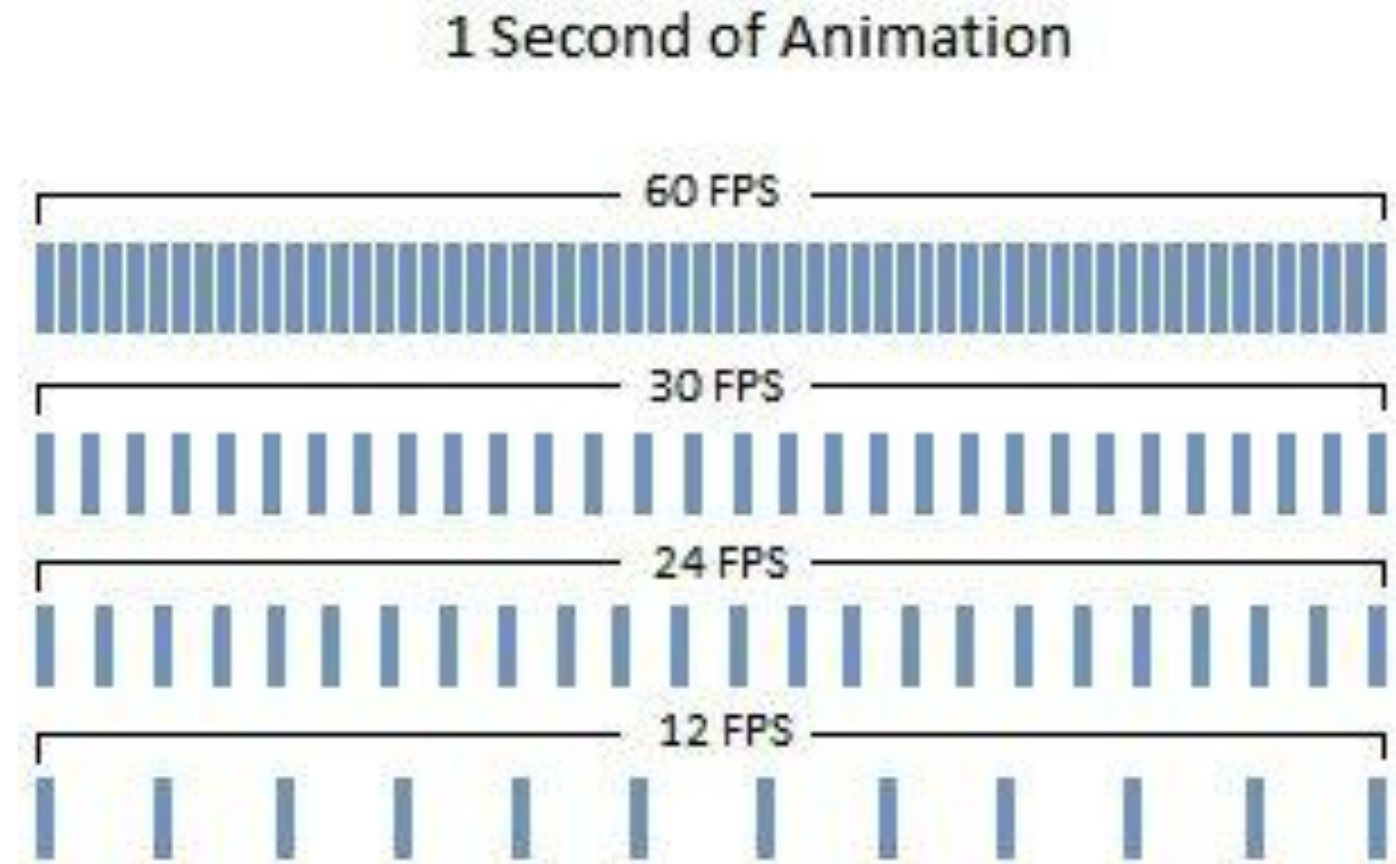


Introduction to Video Processing

Frame rate (fps)?

Frame rate determines the number of "images" seen per second.

Can be described as fps (frames per second) or Hz (general unit for frequency)



Introduction to Video Processing

Video format?

.mp4 (MPEG-4 Part 14)

- Most widely used format
- Excellent compression efficiency with high visual quality
- Ideal for streaming, web sharing, and mobile devices
- Supports video, audio, subtitles, and metadata
- Compatible with most platforms and media players

.avi (Audio Video Interleave)

- Developed by Microsoft
- Supports both audio and video
- Often larger file sizes due to limited compression
- Very compatible across operating systems and tools
- Great for archiving or simple playback

.mov (QuickTime Movie)

- Apple-developed format
- High-quality media used in macOS/iOS environments
- Supports multiple media tracks (audio, video, text)
- Common in video editing and professional workflows
- May require QuickTime codecs on Windows/Linux

.wmv (Windows Media Video)

- Microsoft format known for strong compression
- Suitable for web-based video and streaming
- Best supported on Windows
- May be less compatible on macOS/Linux systems

.mkv (Matroska Video)

- Open-source container format
- Supports multiple audio/video/subtitle streams in one file
- Popular for high-definition video

OpenCV basics

```
#We read the first image and look at the image size
cap = cv2.VideoCapture('005ee183-0100ac18.mp4')
ret, img = cap.read()
print(f'Returned {ret} and img of shape {img.shape}')
cv2_imshow(img)
```


What is object detection?

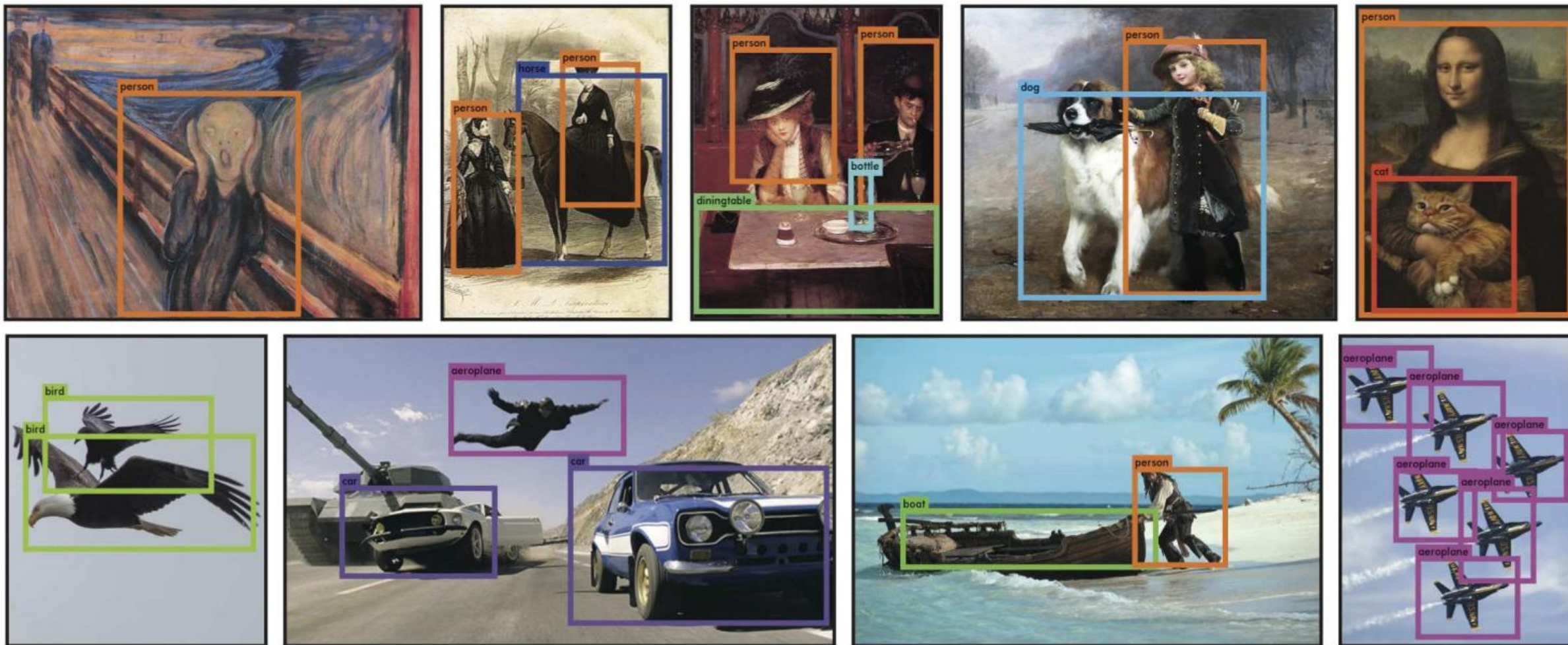
Object Detection refers to **identifying and locating** one or more objects in an image.

Outputs: Bounding boxes with coordinates, class labels, and confidence scores.

Difference between classification and detection?

- Classification: There's a dog in this image.
- Detection: There's a dog at (x=32, y=40, width=200, height=180).

What is object detection?



What is object detection?

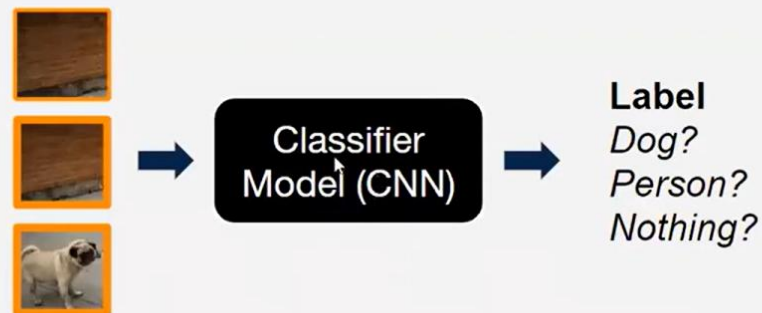
Methods:

Sliding Window + Classifier: Manually scan small regions of the image using a fixed window and apply a classifier. Simple but slow and limited to fixed scales.

1. Scan the image with a sliding window.



2. Feed the images into a classifier model to predict a label for that region.



What is object detection?

Methods:

Sliding Window + Classifier: Manually scan small regions of the image using a fixed window and apply a classifier. Simple but slow and limited to fixed scales.

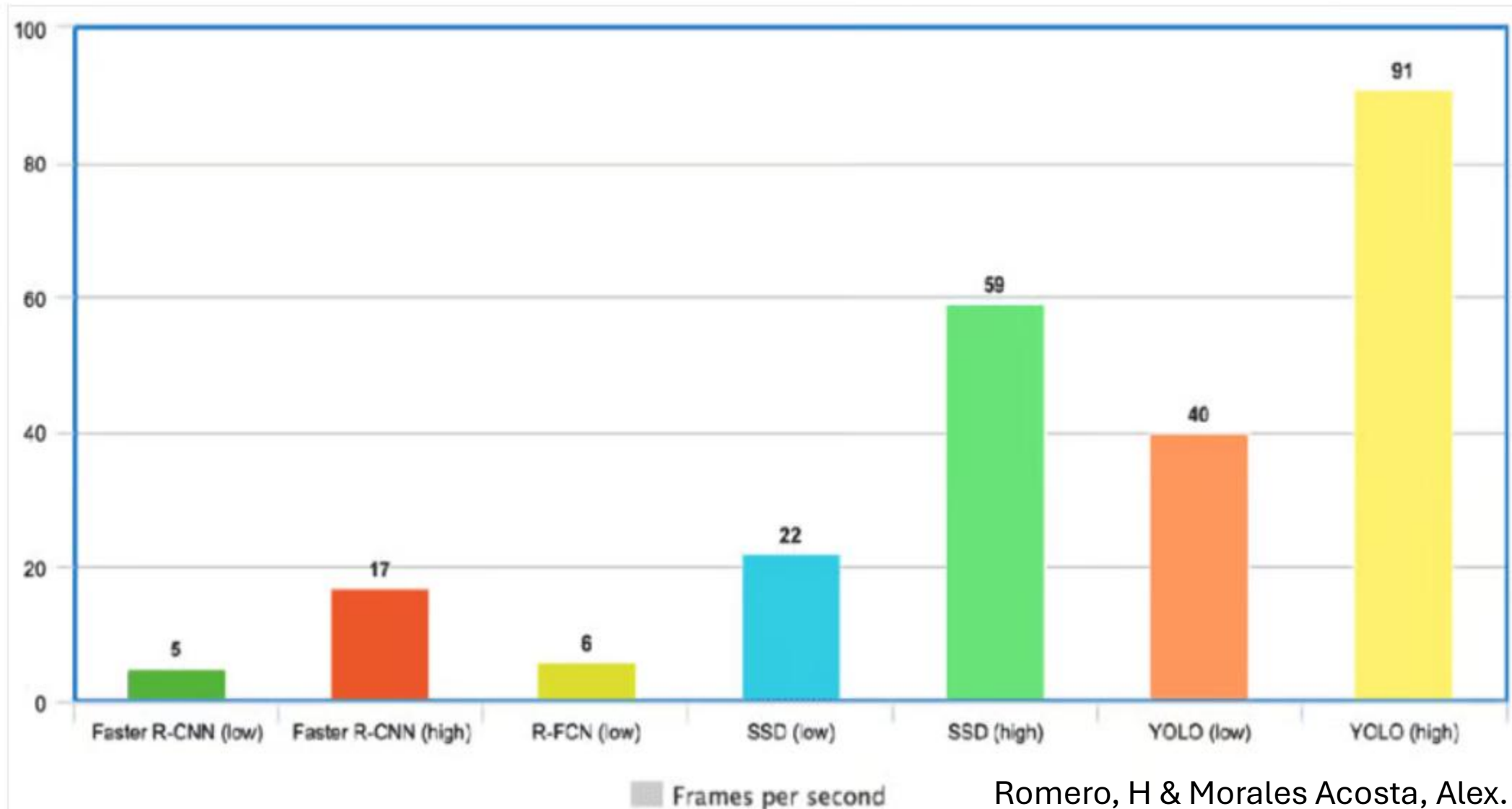
Region-based CNNs :

- R-CNN: Strategically select regions and then classifies each region using CNN features (slow)
- Fast R-CNN: Improves speed by computing CNN features once for the whole image and sharing across regions.

Single-Shot Detectors (e.g. YOLO):

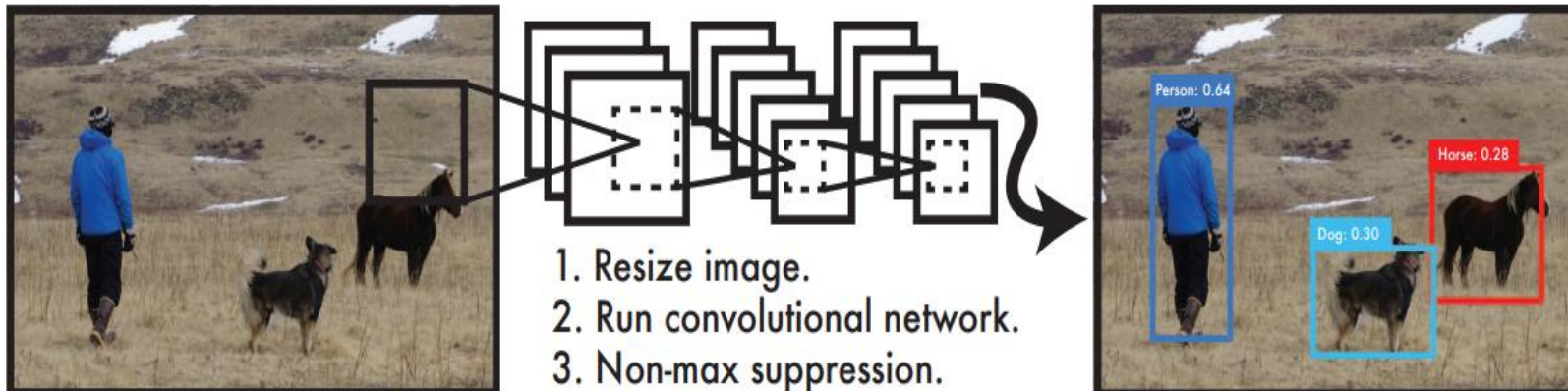
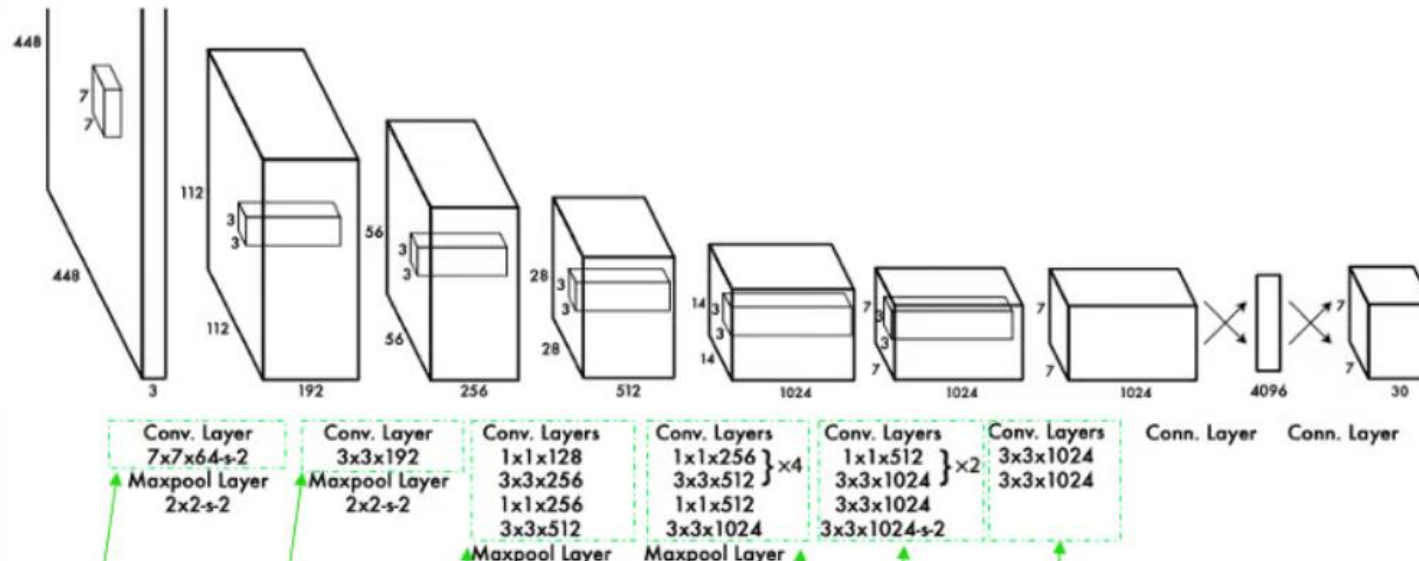
- Predict bounding boxes and class labels directly in one pass through the network.
- Extremely fast (real-time), but sometimes less precise on small objects.

YOLO: You Only Look Once



Romero, H & Morales Acosta, Alex. (2020)

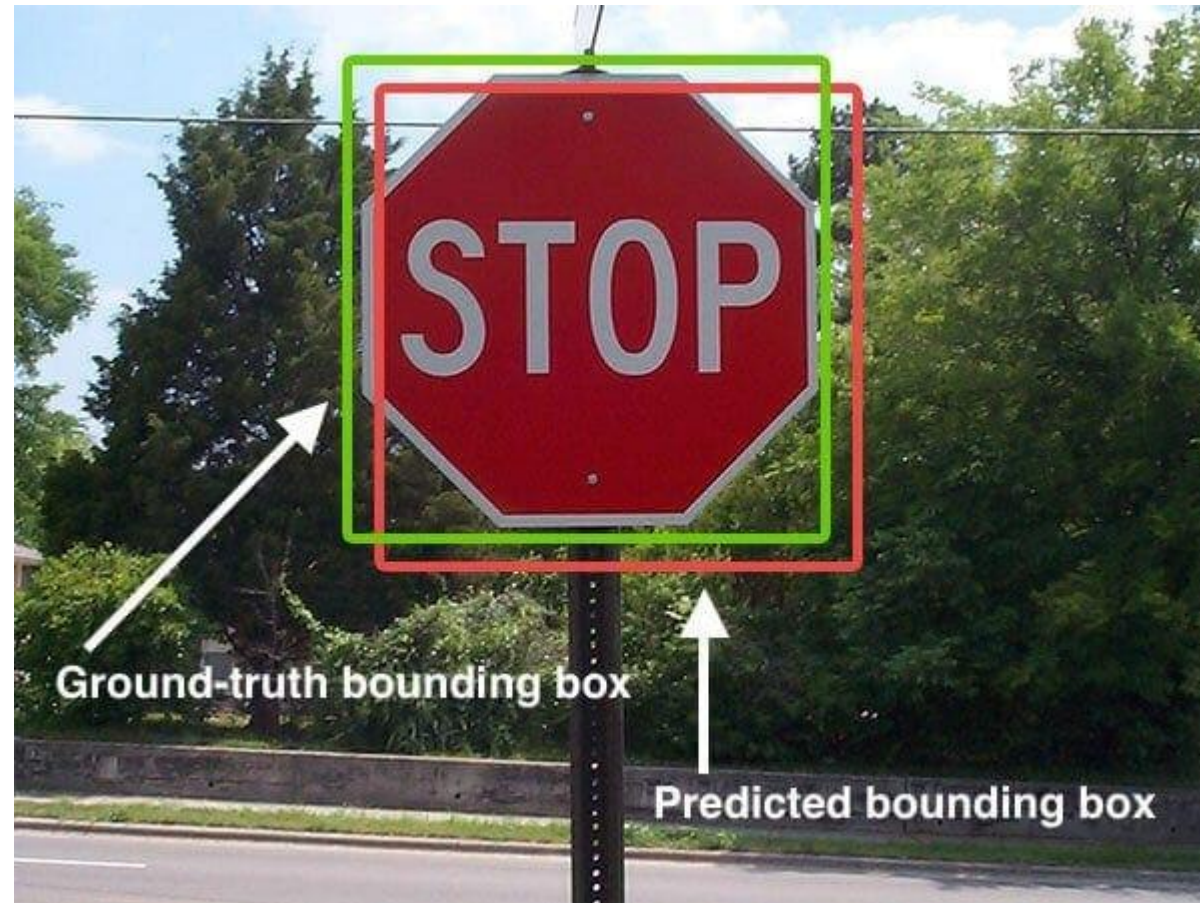
YOLO



- Resizes the input image into 448x448 before going through the convolutional network.
- A 1x1 convolution is first applied to reduce the number of channels, followed by a 3x3 convolution to generate a cuboidal output.
- The activation function under the hood is ReLU, except for the final layer, which uses a linear activation function.
- Some additional techniques, such as batch normalization and dropout, regularize the model and prevent it from overfitting.

YOLO: training

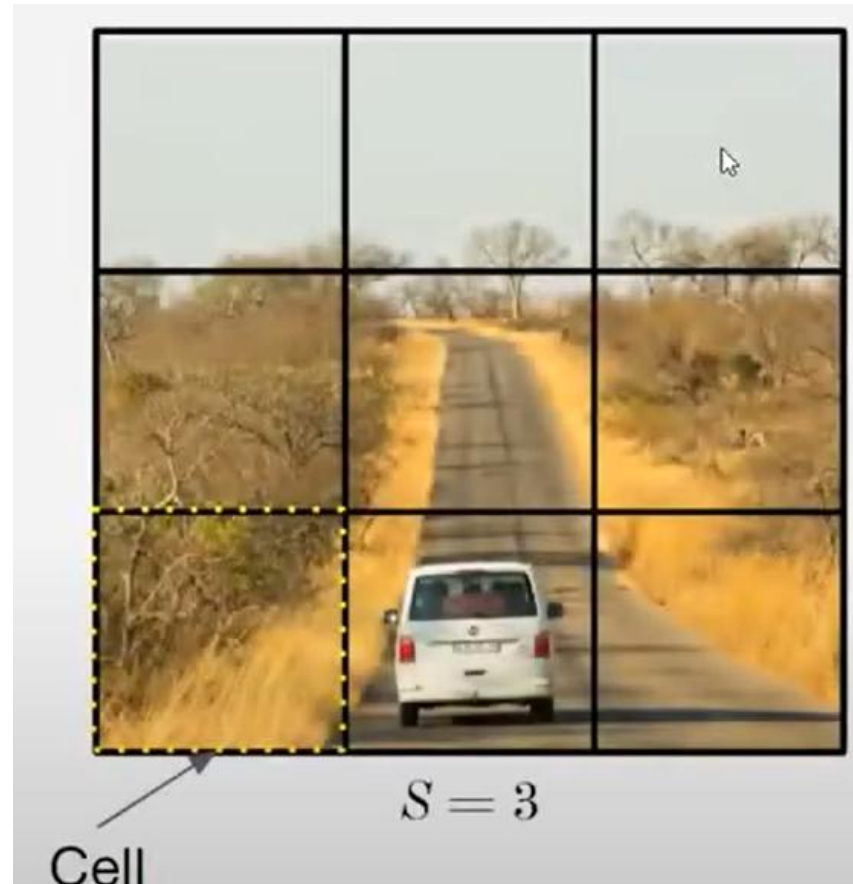
Goal: Teach the model to detect objects by adjusting predicted boxes to match ground truth.



YOLO: training step by step

- Divides image into **$S \times S$ grid**

Each grid cell is responsible for detecting objects whose center falls inside it



YOLO: step by step

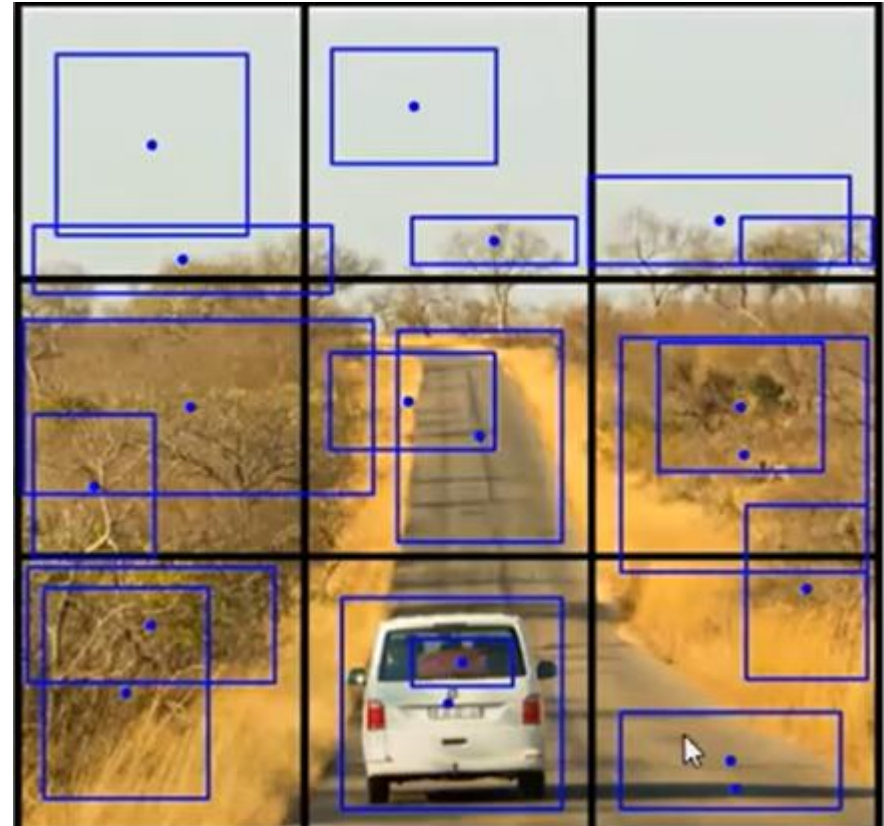
Each Grid Cell Predicts B anchor boxes (predefined shapes)

Each cell has predefined anchor boxes — typical object shapes.

For each anchor box, YOLO predicts:

- Objectness score
- Bounding box offsets:
 - $tx, ty \rightarrow$ Box center (relative to cell)
 - $tw, th \rightarrow$ Box size (relative to anchor)
- Class probabilities

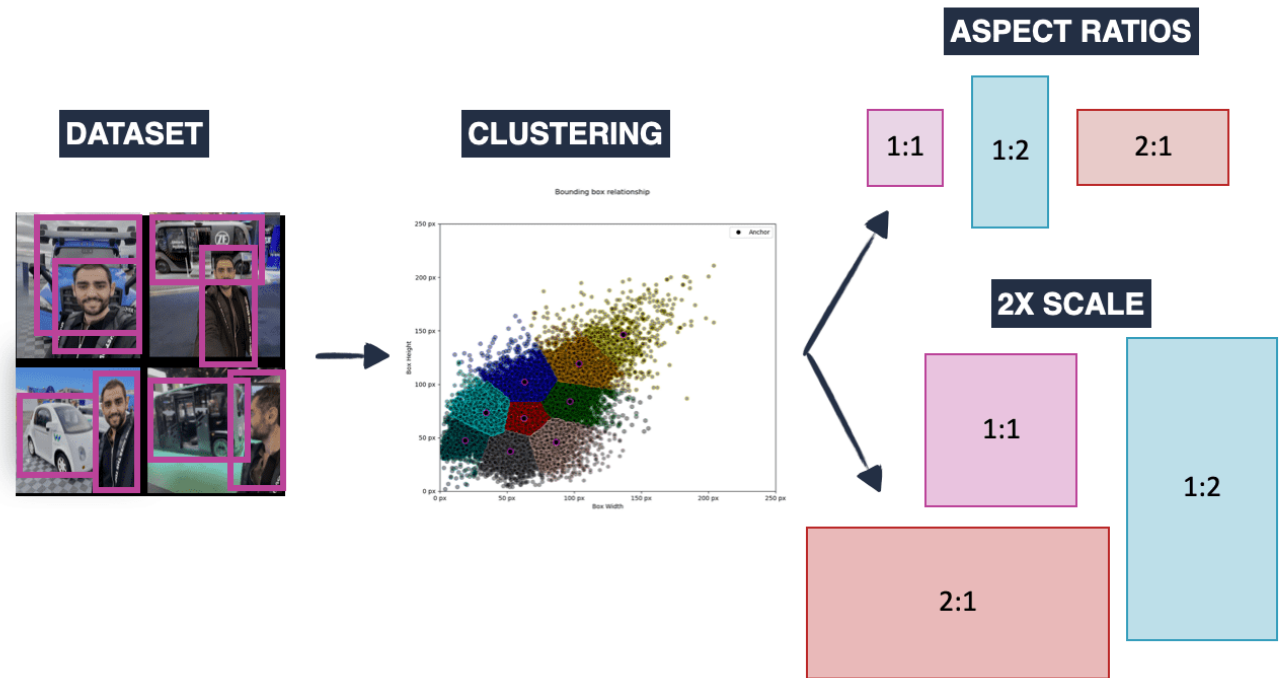
USING ANCHOR BOX ALLOWS TO HAVE MULTIPLE OBJECTS IN THE SAME CELL



YOLO: step by step

Anchor boxes are predefined bounding boxes that serve as reference points for YOLO. They come in different shapes and sizes, strategically chosen to encompass the wide variability of real-world objects.

How anchored box are obtained?



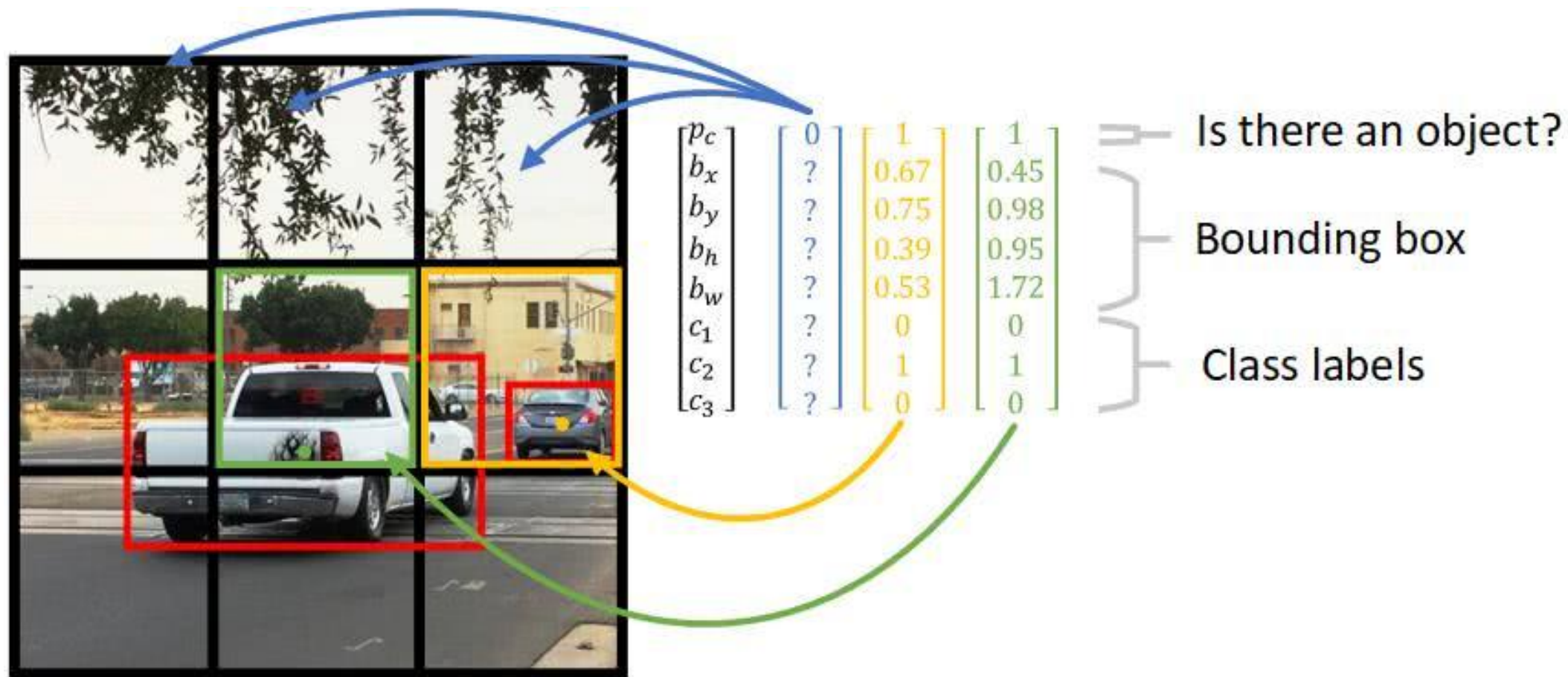
YOLO: step by step

Update the anchored box:

- Compare predicted boxes to ground truth using IoU (Intersection over Union)
- Total loss = localization + objectness + classification
- Localization Loss
Penalizes errors in predicted box center (tx, ty) and size (tw, th) using mean squared error (MSE).
- Objectness Loss
Encourages the model to assign high confidence (objectness score) only if IoU > 0.5 with a ground truth box.
- Classification Loss
Penalizes misclassification (also using MSE or cross-entropy in later versions).

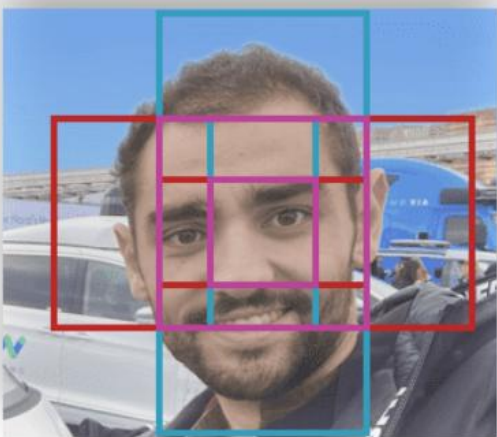

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

YOLO: boundary box



YOLO: boundary box

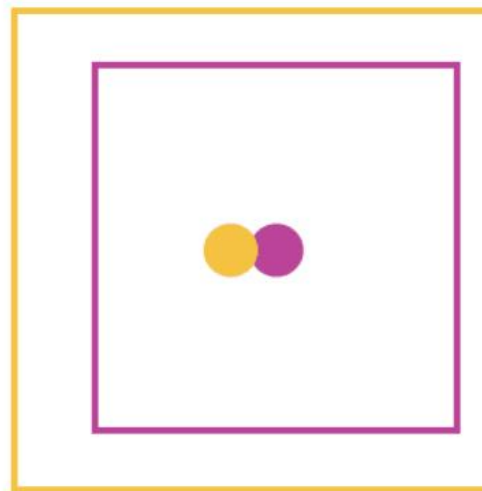
ANCHOR BOXES



KEPT BOX



REFINEMENT



WHAT THE NETWORK PREDICTS

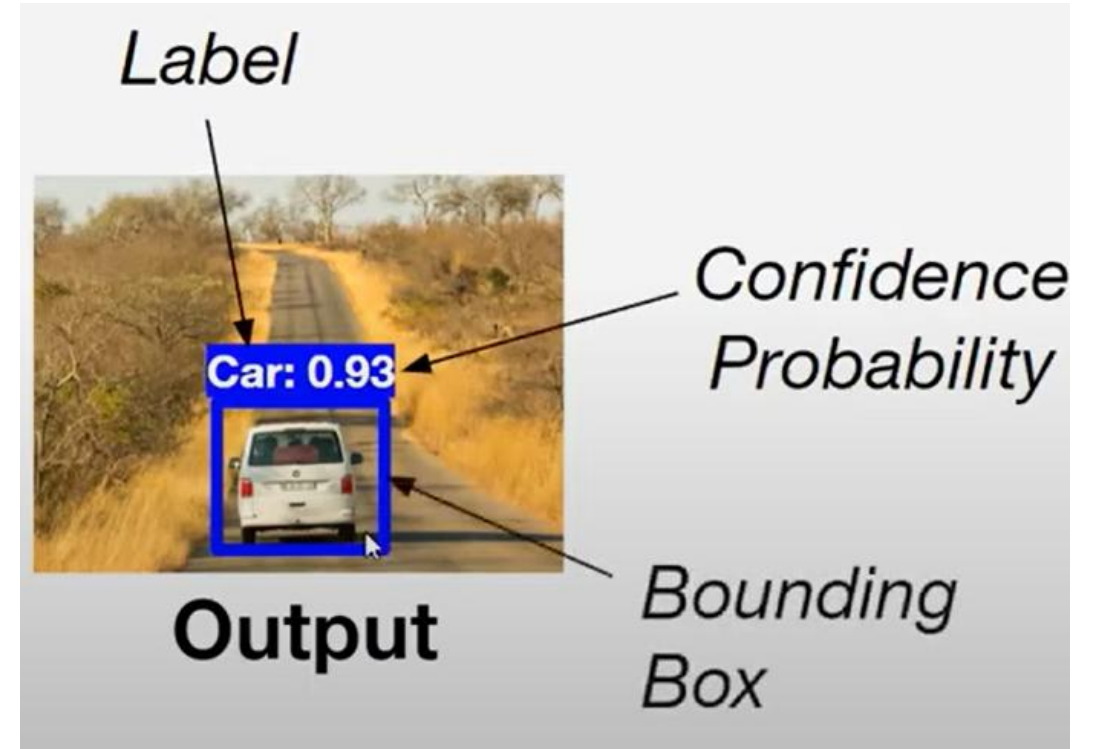
● **center:** Move 5 Pixels left

↑
↓
height: +31 pixels

←
→
width: +26 pixels

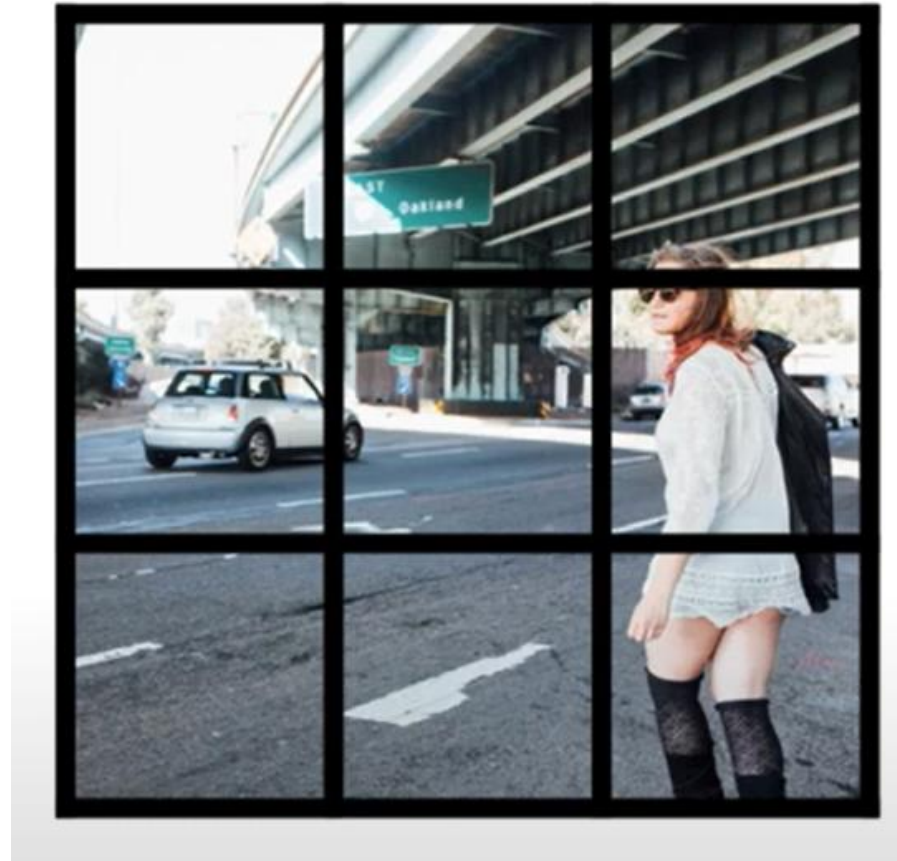
YOLO: inference

Goal: Predict objects in a new image using what the model has learned.



YOLO: inference

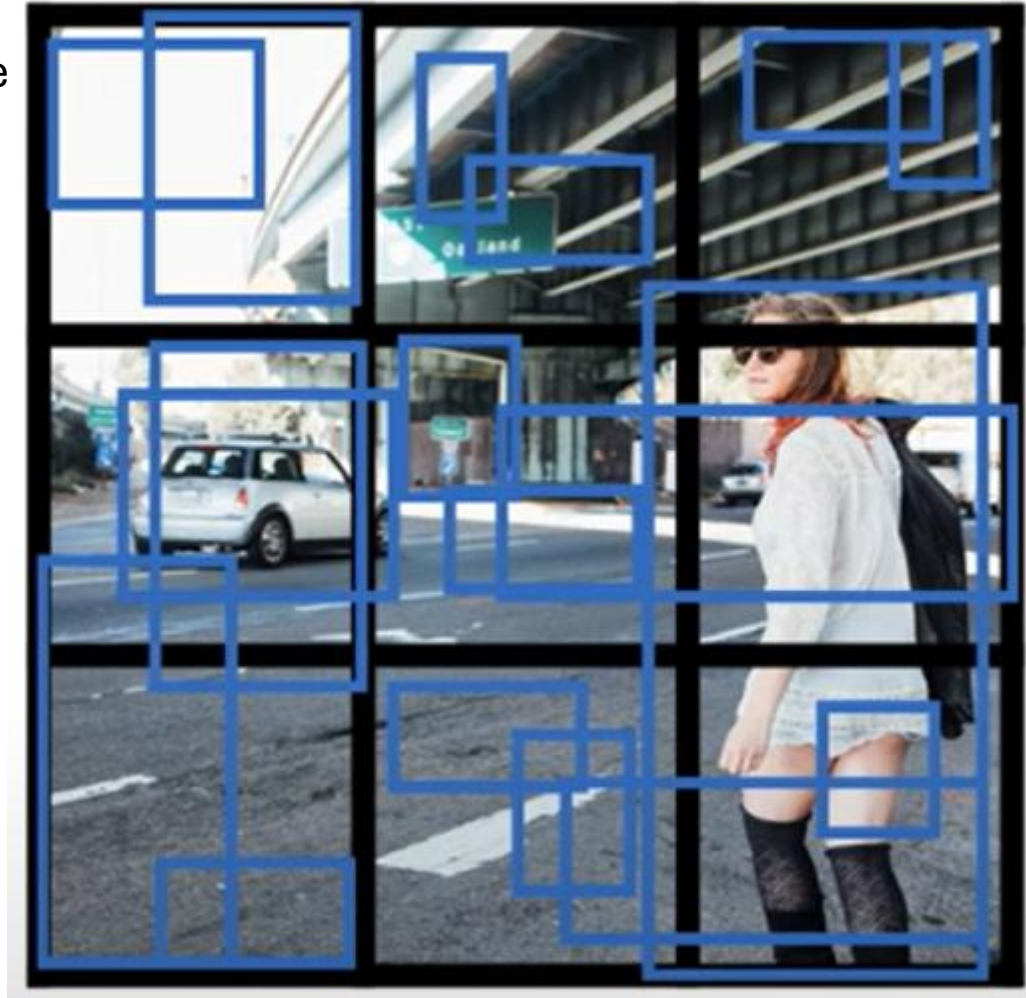
- Divides image into **S x S** grid



YOLO: inference

- Each cell predicts 2 boundary boxes

Note the center of each box should be
Inside 1 cell, but box can overlap
Several cells



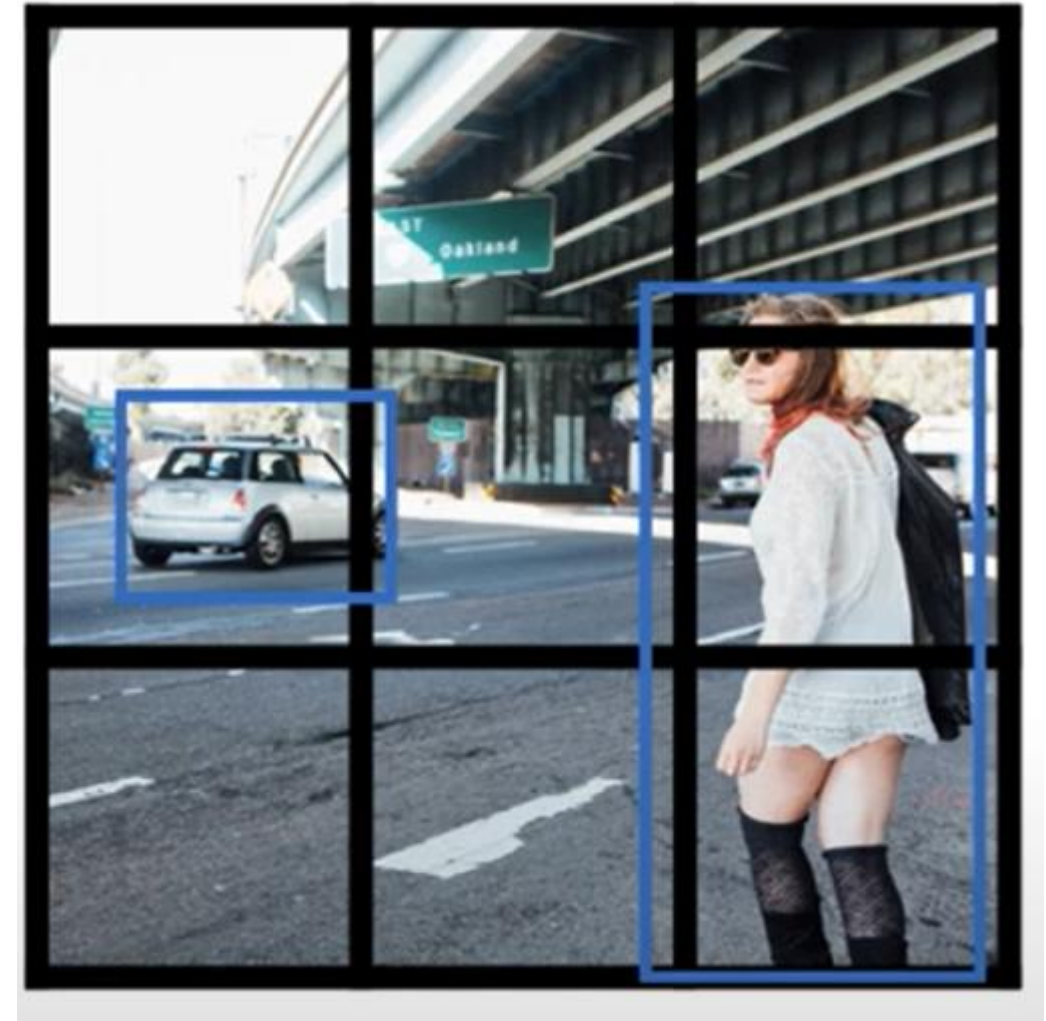
YOLO: inference

- Get ride of low probability



YOLO: inference

- Apply **Non-Maximum Suppression (NMS)**
- Compute IoU between overlapping boxes
- Remove boxes with $\text{IoU} > \text{threshold}$ (i.e., too much overlap)
- Keep the one with highest confidence

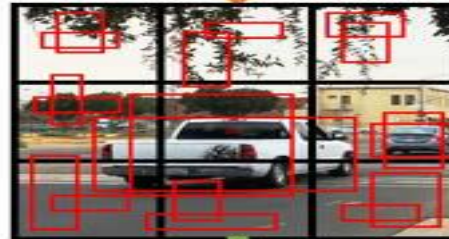


Preprocessed image (600, 600, 3)



Deep CNN
Reduction
Factor: 200

Convert final layer
features to
bounding box
parameters



Filter by
class scores



Non-max
Suppression



YOLO: pretrain model

Is your object you want to detect in this list (COCO list)

person	fire hydrant	elephant	skis	wine glass	broccoli	dining table	toaster
bicycle	stop sign	bear	snowboard	cup	carrot	toilet	sink
car	parking meter	zebra	sports ball	fork	hot dog	tv	refrigerator
motorcycle	bench	giraffe	kite	knife	pizza	laptop	book
airplane	bird	backpack	baseball bat	spoon	donut	mouse	clock
bus	cat	umbrella	baseball glove	bowl	cake	remote	vase
train	dog	handbag	skateboard	banana	chair	keyboard	scissors
truck	horse	tie	surfboard	apple	couch	cell phone	teddy bear
boat	sheep	suitcase	tennis racket	sandwich	potted plant	microwave	hair drier
traffic light	cow	frisbee	bottle	orange	bed	oven	toothbrush

YES

NO

Download pretrained model
Use YOLO (see lab)

Find images of your object
Label them
Train YOLO model (openCV,
Pytorch)

YOLO: limitation

Even though YOLO is fast and powerful, it has some key limitations:

- **Difficulty Detecting Small Objects:**
 - Small objects may fall into multiple grid cells, leading to poor detection accuracy.
- **Fixed Grid Constraint:**
 - Each grid cell can only predict a fixed number of objects (typically 1 or 2). Overlapping or close objects might be missed.
- **Struggles with Complex Backgrounds:**
 - In cluttered scenes or with occlusions, YOLO may confuse objects or miss them entirely.
- **Anchor Box Dependency:**
 - Performance heavily depends on the choice of anchor box shapes. Poorly chosen anchors lead to inaccurate bounding boxes.
- **Less Accurate than Two-Stage Detectors:**
 - Models like Faster R-CNN are slower but typically achieve better precision and recall, especially for dense object scenes.

OCR

OCR

What is OCR?

Optical Character Recognition (OCR) is the process of converting text from scanned images, documents, or scenes into machine-readable digital text.

It allows automation of text entry from physical documents, enabling tasks like:

- Digitizing books
- Reading license plates
- Processing forms or receipts
- Text translation in augmented reality

OCR Pipeline

- **Pre-processing**
 - Convert to grayscale
 - Apply noise removal (Gaussian blur, median filter)
 - Thresholding (Otsu's, adaptive)
 - Skew correction (Hough lines, rotation)
- **Text Detection**
 - Detect regions likely to contain text (bounding boxes with YOLO)
- **Text Recognition**
 - Run character recognition on each detected region (Tesseract OCR, EasyOCR)
- **Post-processing**
 - Remove false positives
 - Clean text using regular expressions, spell check
 - Language filtering, formatting

OCR

Problem

Low contrast

Noise / blur

Rotated text

Complex fonts

Multilingual docs

Solution

Histogram equalization, adaptive threshold

Denoise filters (Gaussian, bilateral)

Deskew using image moments or Hough transform

Fine-tune models or use deep OCR (TrOCR)

Use language models or multiple OCR passes